Quantum Machine Learning with Quantum Kernels

Why and how to build a Quantum Support Vector Machine

Q Hackathon pre-training · QML module

Outline

1. **Why** a quantum SVM?
2. Background: qubits and measurement
3. Machine-learning basics: evaluation, feature maps, the kernel trick, SVMs
4. Putting data into qubits: **quantum feature maps**
5. How a quantum computer measures a **kernel**
6. QSVM on IBM's *ad hoc* dataset
7. Pitfalls, QSVM vs. VQC, real hardware
8. Hackathon challenge: **is this element a metal?**

After this lecture you can

- explain what a kernel and an SVM are
- build a quantum feature map in Qiskit
- compute a quantum kernel, exactly and with shots
- train a QSVM and compare it fairly with a classical SVM

Tools: **Qiskit**, **Qiskit Machine Learning**, **scikit-learn**. Runnable version: `QML_1_Lecture.ipynb`.

Why a quantum SVM?

Why are we doing quantum SVMs?

1. A quantum computer is a natural "kernel machine"

- n qubits = a 2^n-dimensional feature space
- The similarity of two data points is an **overlap of quantum states**, measured directly as a probability

2. Kernels that may be hard to compute classically

- Entangling feature maps are believed hard to simulate (Havlíček *et al.*, *Nature* 2019, IBM)
- Proven speed-up for specially built problems (Liu, Arunachalam & Temme, *Nat. Phys.* 2021)

3. Training stays classical and *convex*

- The QPU only estimates kernel entries; a standard SVM does the learning
- Global optimum guaranteed: **no local minima, no barren plateaus, no optimiser loop on the QPU** (unlike VQE / VQC)

4. Near-term friendly

- Short, shallow circuits; just count $00\ldots0$ outcomes

Honest point of view : a quantum *advantage* on real-world data is still an **open question**. Today: learn the method and how to test it fairly.

Background: qubits in one slide

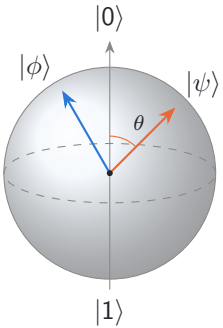
Qubits, measurement and gates: what we need today

- A qubit state: $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$, $|\alpha|^2 + |\beta|^2 = 1$
- **Measurement**: 0 with probability $|\alpha|^2$, 1 with $|\beta|^2$; repeating it (**shots**) estimates them
- Every state is a point on the **Bloch sphere**:

$$|\psi\rangle = \cos\tfrac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\tfrac{\theta}{2}\,|1\rangle$$

- **Gates** are rotations: $R_y(\theta)$ tilts the state, H creates superposition, **CX** entangles two qubits
- n qubits $\Rightarrow 2^n$ amplitudes: an **exponentially large** vector space

Overlap $|\langle\phi|\psi\rangle|^2$: 1 for identical states, 0 for orthogonal ones (opposite poles). This will be our **similarity**.

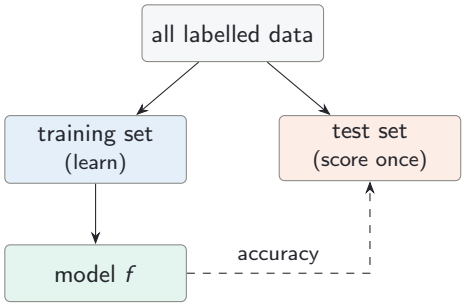


Machine-learning basics

Classification: learn a rule from labelled examples

- Data points $\boldsymbol{x}$ = lists of numbers (*features*)
- Each has a **label** $y \in \{0, 1\}$
  e.g. non-metal / metal
- Learn $f(\boldsymbol{x}) \to y$ on a **training set**
- Judge it **only** on an unseen **test set**

A model that memorises its training data is useless. **Test accuracy is the only honest score.**



How do we judge a classifier?

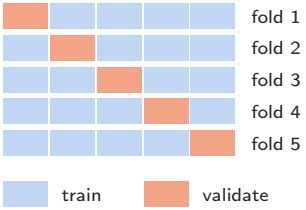
Confusion matrix (test set)

	predicted 0	predicted 1
true 0	**TN** correct	**FP** wrong
true 1	**FN** wrong	**TP** correct

- **Accuracy** = fraction correct
- **Balanced accuracy** = average of the accuracy on each class

90% of the data in one class? "Always predict that class" scores **90% accuracy** but only **50% balanced accuracy**.

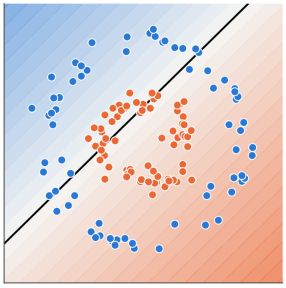
Cross-validation (CV): choose settings without touching the test set



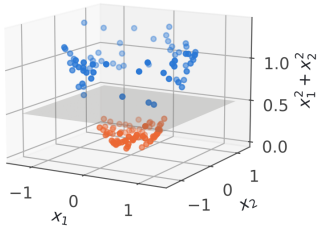
- Split the **training** set into 5 folds; train on 4, score on 1; repeat 5× and average
- Use the CV score to pick hyperparameters (C, γ, later the bandwidth λ)
- Use the **test set once**, at the very end

A straight line is not enough → use a feature map

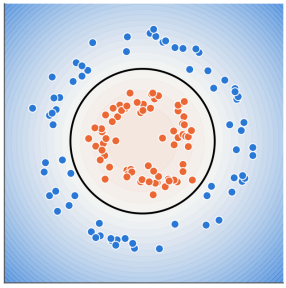
Straight line: 58% test accuracy



Feature space: a flat plane separates



Same linear SVM after ϕ: 100%



$$\boldsymbol{x} = (x_1, x_2) \longmapsto \phi(\boldsymbol{x}) = \left(x_1,\ x_2,\ x_1^2 + x_2^2\right)$$

Key idea of the lecture: map the data into a **bigger space** where a flat plane separates it.
In QML the bigger space is the **Hilbert space of qubits**, which is exponentially big.

The kernel trick and the support vector machine (SVM)

- SVM: the separating plane with the **widest margin**
- Its maths uses the data **only through inner products**. Call them the kernel:

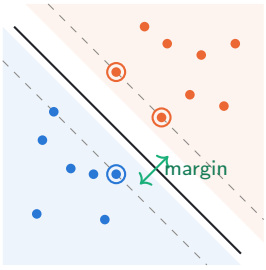
$$k(\boldsymbol{x}, \boldsymbol{x}') = \phi(\boldsymbol{x}) \cdot \phi(\boldsymbol{x}') \quad \text{(similarity)}$$

- Prediction:

$$f(\boldsymbol{x}) = \operatorname{sign}\Big(\sum_{i \in \text{train}} \alpha_i \, y_i \, k(\boldsymbol{x}_i, \boldsymbol{x}) + b \Big)$$

only the **support vectors** have $\alpha_i \neq 0$

Training an SVM is a **convex** problem: the solver always finds the global optimum.



circled = support vectors

Inside the SVM: the optimisation problem

With labels $y_i \in \{-1, +1\}$, the SVM solves (dual form):

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j \, y_i y_j \, K_{ij}$$

$$\text{subject to} \quad 0 \le \alpha_i \le C, \qquad \sum_i \alpha_i y_i = 0$$

- The data appear **only** through $K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$
- A quadratic problem: if K is a valid kernel (positive semidefinite), it is **convex** $\Rightarrow$ one global optimum, found reliably
- Points with $\alpha_i > 0$ are the **support vectors**

The hyperparameter C

- **small** C: wide margin, tolerates some training errors (smoother, less overfitting)
- **large** C: narrow margin, tries to classify every training point (risk of overfitting)
- choose C by cross-validation

Compare with VQE: there the cost landscape has local minima. Here the hard part (the kernel) is **computed**, not **optimised**.

The SVM only needs a kernel *matrix*

1. Compute $K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$ for all training pairs
2. Hand K to an ordinary SVM solver
3. For new points: their kernel with the training points

Step 1 is the only place the feature map appears.

If a quantum computer can estimate k for a feature map that is hard classically, we plug it into a *completely classical* SVM.

That is the Quantum SVM (QSVM).

```python
from sklearn.svm import SVC

# K_train[i, j] = k(x_i, x_j)        (N x N)
# K_test[a, j]  = k(x_test_a, x_j)   (M x N)

svm = SVC(kernel="precomputed")
svm.fit(K_train, y_train)

y_pred = svm.predict(K_test)
print(svm.score(K_test, y_test))
```

Where K comes from (classical formula or quantum computer) doesn't matter to the SVM.

Quantum feature maps

How can classical data enter a quantum computer?

encoding	**idea**	**qubits for d features**	**good to know**
basis	bit string $\to \lvert 0110\rangle$	d (per bit)	only for binary data
angle	one rotation per feature, $R_y(x_i)\lvert 0\rangle$	d	simple, shallow; kernel is a product $\to$ classically easy
amplitude	features $\to$ the 2^n amplitudes	$\log_2 d$	very compact, but deep state-preparation circuits
ZZ / IQP	angles + entangling phases $x_i x_j$	d	entangled; kernel believed **hard to simulate** classically

Today: **angle encoding** to build intuition, then IBM's **ZZ feature map** for a QSVM.

The encoding *is* the model: a quantum kernel is only as good as its feature map.

Putting data into qubits

A **quantum feature map** is a circuit whose rotation angles are the data:

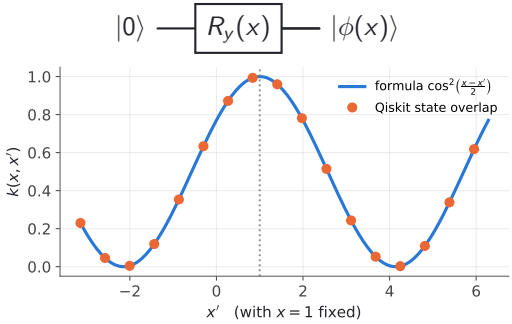
$$\boldsymbol{x} \longmapsto |\phi(\boldsymbol{x})\rangle = U(\boldsymbol{x})\,|0\ldots0\rangle$$

The **quantum kernel** is the fidelity of two states:

$$\boxed{k(\boldsymbol{x}, \boldsymbol{x}') = \left|\langle\phi(\boldsymbol{x}')|\phi(\boldsymbol{x})\rangle\right|^2}$$

One qubit: $R_y(x)\,|0\rangle = \cos\frac{x}{2}\,|0\rangle + \sin\frac{x}{2}\,|1\rangle$

$$\Rightarrow\ k(x, x') = \cos^2\left(\frac{x - x'}{2}\right)$$



Rotations are periodic: **always scale features to** $[0, \pi]$.

Worked example: a quantum kernel matrix by hand

Three data points, one qubit, $|\phi(x)\rangle = R_y(x)\,|0\rangle$:

$$x_1 = 0, \qquad x_2 = \tfrac{\pi}{2}, \qquad x_3 = \pi$$

- $|\phi(0)\rangle = |0\rangle$ (north pole)
- $|\phi(\pi/2)\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ (equator)
- $|\phi(\pi)\rangle = |1\rangle$ (south pole)

With $k(x, x') = \cos^2\left(\frac{x-x'}{2}\right)$:

$$K = \begin{pmatrix} 1 & 0.5 & 0 \\ 0.5 & 1 & 0.5 \\ 0 & 0.5 & 1 \end{pmatrix}$$

```python
import numpy as np
from qiskit import QuantumCircuit
from qiskit.circuit import Parameter
from qiskit_machine_learning.kernels import \
    FidelityStatevectorKernel

x = Parameter("x")
fm = QuantumCircuit(1); fm.ry(x, 0)
kernel = FidelityStatevectorKernel(feature_map=fm)
X = np.array([[0], [np.pi/2], [np.pi]])
print(kernel.evaluate(X).round(3))
# [[1.  0.5 0. ]
#  [0.5 1.  0.5]
#  [0.  0.5 1. ]]
```

Always: **symmetric**, **diagonal = 1**, and positive semidefinite (it is an inner product) $\Rightarrow$ a valid SVM kernel.

IBM's ZZ feature map: entanglement makes it interesting

Without entangling gates, $k = \prod_i \cos^2 \frac{x_i - x_i'}{2}$: **classically easy**. The ZZ map makes features *interact*:

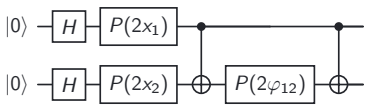
$$U(\boldsymbol{x}) = \exp\Big(i \sum_i x_i Z_i + i \sum_{i<j} \varphi_{ij}\, Z_i Z_j\Big) H^{\otimes n}$$

$\varphi_{ij} = (\pi - x_i)(\pi - x_j)$, repeated `reps` times

```
from qiskit.circuit.library import zz_feature_map

fmap = zz_feature_map(feature_dimension=2, reps=2)
fmap.draw("mpl")
```

one layer, 2 qubits:



- **H**: superposition
- **P**$(2x_i)$: encode each feature
- **CX–P–CX**: encode *products* of features → entanglement

Measuring a quantum kernel

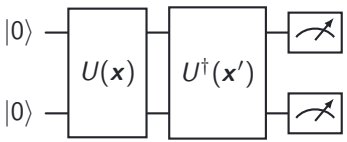
How a quantum computer measures $k(x, x')$

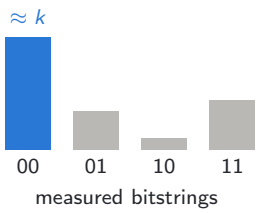
Hardware gives no wavefunctions, only measurement **shots**. Use the compute–uncompute circuit:

$$P(0\ldots0) = \left| \langle 0| U^\dagger(x') U(x) |0\rangle \right|^2 = k(x, x')$$

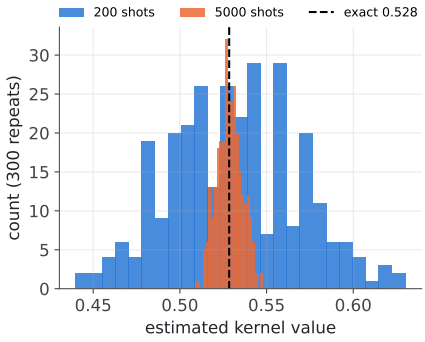
Kernel entry = fraction of shots that read $00\ldots0$.
If $x = x'$, the second half undoes the first
$\Rightarrow k = 1$.

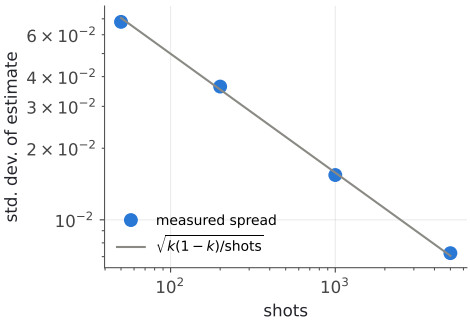
Example: 2000 shots $\rightarrow$ 1059 times "00"
$\Rightarrow k \approx 0.530$ (exact 0.528)





The price: shot noise and N^2 circuits

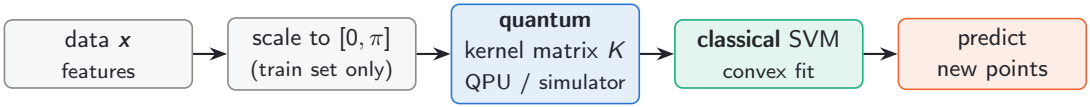




Noise $\propto 1/\sqrt{\text{shots}}$: **4× more shots for 2× less noise** (the same bottleneck as VQE).

train N	train circuits $\frac{N(N-1)}{2}$	per 100 test pts
40	780	4 000
200	19 900	20 000
1000	499 500	100 000

The complete QSVM pipeline (Qiskit Machine Learning)




```python
from qiskit.circuit.library import zz_feature_map
from qiskit.primitives import StatevectorSampler
from qiskit_machine_learning.kernels import FidelityQuantumKernel
from qiskit_machine_learning.state_fidelities import ComputeUncompute
from qiskit_machine_learning.algorithms import QSVC

fmap    = zz_feature_map(feature_dimension=2, reps=2)
sampler = StatevectorSampler(default_shots=1000)    # later: a real IBM backend
qkernel = FidelityQuantumKernel(feature_map=fmap, fidelity=ComputeUncompute(sampler=sampler))

qsvc = QSVC(quantum_kernel=qkernel).fit(X_train, y_train)   # SVC + quantum kernel matrix
print("test accuracy:", qsvc.score(X_test, y_test))
```

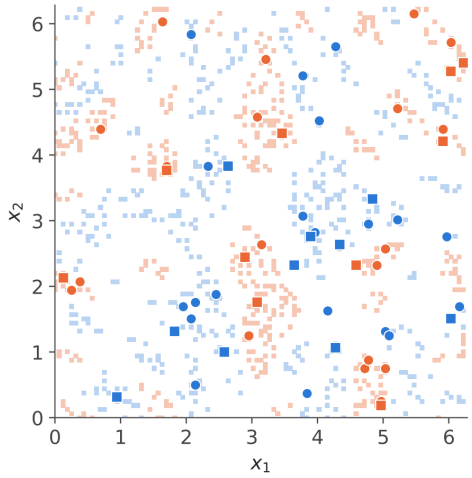
Fast exact version for laptops: `FidelityStatevectorKernel(feature_map=fmap)`

QSVM on IBM's ad hoc dataset

A dataset built for the ZZ feature map

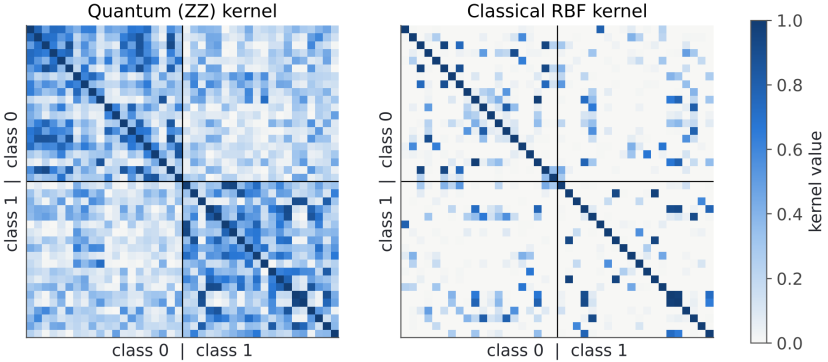
- From Havlíček *et al.* (IBM, 2019); in Qiskit as `ad_hoc_data`
- Labels defined **through the ZZ feature map** itself (a hidden observable), with a gap around the boundary
- 40 training points, 20 test points, 2 features
- Background: 1000 more points from the same rule. A complicated pattern

Hard for standard classical kernels, **natural** for the quantum one.



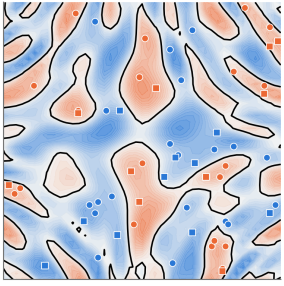
dots: training · squares: test

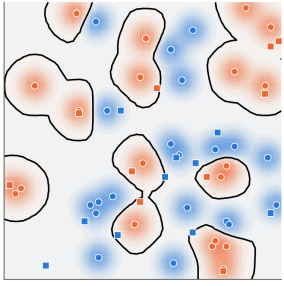
Kernel matrices: quantum vs. classical



Sorted by class, a good kernel is **block-diagonal**: similar *within* a class, dissimilar *between* classes. The quantum kernel shows the class structure; the RBF kernel only sees nearby points.

Result: the quantum kernel wins (on this dataset)





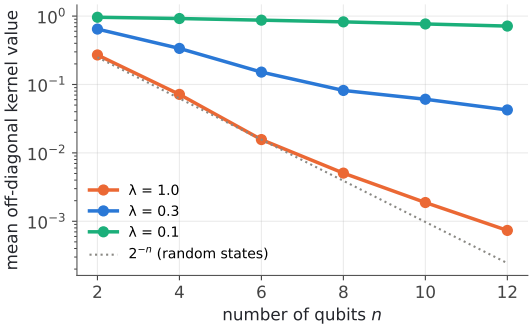
	test (20 pts)	1000 extra points
QSVM (ZZ)	**1.00**	**1.00**
RBF SVM (tuned)	0.90	0.69

- QSVM draws the right *kind* of shapes
- RBF draws blobs around training points

The data was **designed** for this feature map. Shows quantum kernels *can* win, **not** that they always do.

Pitfalls, alternatives, hardware

Pitfall: more qubits $\neq$ better $\rightarrow$ tune the bandwidth λ



mean off-diagonal kernel value for random data; Thanasilp *et al.*, *Nat. Commun.* 2024

- In a 2^n-dimensional space, random states are **almost orthogonal**: kernel $\rightarrow$ identity, the SVM can only memorise (exponential concentration)
- **Cure:** scale the data by a bandwidth λ before encoding

$$k_\lambda(\boldsymbol{x}, \boldsymbol{x}') = \left|\langle\phi(\lambda\boldsymbol{x}')|\phi(\lambda\boldsymbol{x})\rangle\right|^2$$

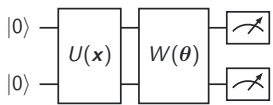
small λ: smooth large λ: memorises

- Choose λ by **cross-validation**, like γ in an RBF kernel (Shaydulin & Wild 2022)

Health check: plot a histogram of your kernel values. All ≈ 0? Trouble.

QSVM vs. variational quantum classifier (VQC)

VQC = feature map + trainable ansatz $W(\boldsymbol{\theta})$ + classical optimiser. The same loop as **VQE**.



	QSVM	VQC
trained	classical SVM	circuit angles $\boldsymbol{\theta}$
optimisation	**convex**, global optimum	non-convex, local minima, barren plateaus
QPU cost	$\sim N^2$ circuits, run once	iterations $\times N$
large N	expensive	scales better
pitfall	concentration	trainability

Both need a **good feature map**. VQC is in the notebook (Part 7) as self-study.

From simulator to a real IBM quantum computer

Only three things change: **backend**, **transpile**, **sampler**.

```
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2
from qiskit.transpiler import generate_preset_pass_manager

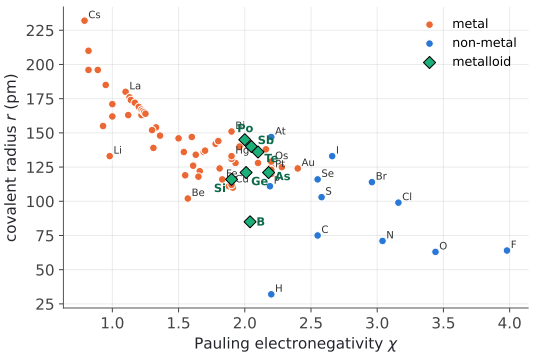
service = QiskitRuntimeService()                              # free IBM Quantum account
backend = service.least_busy(operational=True, simulator=False)
pm      = generate_preset_pass_manager(optimization_level=3, backend=backend)

sampler = SamplerV2(mode=backend); sampler.options.default_shots = 1000
qk_hw   = FidelityQuantumKernel(feature_map=pm.run(zz_feature_map(2, reps=2)),
                                fidelity=ComputeUncompute(sampler=sampler, pass_manager=pm))
K_hw = qk_hw.evaluate(X_train[:10])                           # 45 circuits on the QPU
```

On hardware, noise pulls $P(00\ldots0)$ down: kernel values come out **too small** and the diagonal < 1. Keep circuits shallow (`reps=1`) and datasets tiny.

Hackathon challenge

Hackathon challenge: Is this element a metal?



Real materials data, `elements_dataset.csv`:

- 76 elements (H–Rn, no noble gases)
- features: electronegativity χ, covalent radius r, density, electron affinity
- label `class`: **57 metals, 12 non-metals**, 7 **metalloids** (the puzzle)

Your questions

- Can a QSVM tell metals from non-metals?
- Is it better than a tuned classical SVM?
- What does it say about the metalloids?

Unlike *ad hoc*, this data was **not** built for any quantum kernel. That's the real test.

Take-aways

1. An SVM only needs a **kernel** (a similarity), and training is **convex**
2. A **quantum feature map** turns data into states; the kernel is their **overlap**, measured as $P(00\ldots0)$
3. **QSVM** = quantum kernel matrix + ordinary classical SVM
4. Scale features, tune the **bandwidth** λ by cross-validation, check the kernel-value histogram
5. Quantum kernels **can** win when the data fits the feature map; on real data, **find out yourself** and test it fairly

Try it yourself: open `QML_1_Lecture.ipynb` in Google Colab → Runtime → Run all.

Questions?

Thank you!

References

- V. Havlíček *et al.*, "Supervised learning with quantum-enhanced feature spaces", *Nature* **567**, 209 (2019).
- M. Schuld & N. Killoran, "Quantum machine learning in feature Hilbert spaces", *PRL* **122**, 040504 (2019).
- Y. Liu, S. Arunachalam & K. Temme, "A rigorous and robust quantum speed-up in supervised machine learning", *Nat. Phys.* **17**, 1013 (2021).
- S. Thanasilp *et al.*, "Exponential concentration in quantum kernel methods", *Nat. Commun.* **15**, 5200 (2024).
- R. Shaydulin & S. M. Wild, "Importance of kernel bandwidth in quantum machine learning", *PRA* **106**, 042407 (2022).
- Qiskit Machine Learning: `https://qiskit-community.github.io/qiskit-machine-learning/`
- QC4MC Summer School 2025, QML notebook: `https://github.com/qc4mc/qc4mcsummer2025/tree/main/QML`